Linked List

A linked list is a linear data structure where elements, called nodes, are not stored in contiguous memory locations. In a linked list, the linear order is maintained using links or pointers. Each node in a linked list contains at least two parts: a data part and an address part (a pointer) that stores the address of the next node.

Core Components

Node: The basic building block that consists of data and at least one address link.

Head: A special pointer to the first node that marks the beginning of the list. Simply, we can say identifies the first node at the starting of list. If the list is empty, the head is null.

Data: Stores the actual value (integer, string, etc.) to be stored and processed.

Next / Link / Pointer: Stores the address or reference of the next node in the list.

Tail: The last node, which typically points to null to signify the end of the list and does not point to any further node.

Basic Operations on Linked List

A linked list supports several fundamental operations that allow efficient data manipulation. These operations are:

1. Creation

This operation involves creating a linked list by allocating memory for nodes and linking them together. Each node contains data and a pointer to the next node.

2. Traversal

Traversal means visiting each node of the linked list starting from the head node and moving sequentially until the last node (NULL) is reached.

3. Insertion

Insertion is the process of adding a new node to the linked list. It can be performed at:

- Beginning
- End
- Any specific position

4. Deletion

Deletion involves removing a node from the linked list. It can be done:

- From the beginning
- From the end
- From a specific position

5. Display

This operation is used to print or display all the elements (data) of the linked list by traversing from the head node to the last node.

6. Search

Search operation is used to find a specific element in the linked list by traversing node by node and comparing values.

Linked List as an ADT/ Types of Linked List Operations

There are several operations that can be performed on a linked list. They are:

1. insert( ) : The insert( ) operation is used to add a new node into the linked list. Insertion can be performed in different ways:

- At the beginning
- At the end
- At a specific position

During insertion, a new node is created and its link is adjusted to maintain the connection between nodes.

2. delete( ) : The delete( ) operation is used to remove a node from the linked list. Deletion can be performed:

- From the beginning
- From the end
- From a specific position

While deleting, the links are updated, and the removed node is freed from memory.

3. traverse( ) / display( ) : The traverse( ) or display( ) operation is used to visit and print all elements of the linked list from the head to the last node.

4. search( ) : The search( ) operation is used to find whether a particular element exists in the linked list or not. It scans each node sequentially until the element is found or the list ends.

5. isEmpty( ) : The isEmpty( ) operation checks whether the linked list is empty or not.

If the head is equal to NULL, the linked list is empty and the operation returns true. Otherwise, it returns false, indicating that the list is not empty

6. length( ) / count( ) : The length( ) operation counts the number of nodes present in the linked list by traversing the entire list.

Types of linked lists

There are three types of lists in data structures:

1. Singly linked list
2. Doubly linked list
3. Circular linked list

Singly Linked List

A singly linked list is a linear data structure in which each element, called a node, contains two parts: a data part and an address (link) part. The address part stores the reference to the next node in the list.

In this linked list, each node points only to the next node in a single direction. The first node is accessed using a pointer called the head, and the last node points to NULL, indicating the end of the list.

In this linked list, traversal is possible only in one direction, i.e., from the head node to the last node (NULL), which is called one-way traversal.

Thus, the nodes are connected sequentially, forming a chain-like structure.

Node Insertion Operation in Singly Linked List (SLL)

1. Insert at Beginning/ Front.
2. Insert at End.
3. Insert after a Particular Position
4. Insert at k-th Position/ at a Specified Position.

Algorithm to insert a node at the beginning of a SLL

```
struct Node {
   int data;
   struct Node *next;
};
struct Node *head = NULL, *newNode;
void insertAtBeginning() {
   newNode = (struct Node *) malloc(sizeof(struct Node));
   printf("Enter data to insert: ");
   scanf("%d", &newNode->data);
   newNode->next = head;
   head = newNode;
}
```

Algorithm to insert a node at the end of a SLL

```
struct Node {
   int data;
   struct Node *next;
};
struct Node *head = NULL, *newNode, *temp;
void insertAtEnd() {
   newNode = (struct Node *) malloc(sizeof(struct Node));
   printf("Enter data to insert: ");
   scanf("%d", &newNode->data);
   newNode->next = NULL;
   if (head == NULL) {
      head = newNode;
   } else {
      temp = head;
      while (temp->next != NULL) {
         temp = temp->next;
      }
      temp->next = newNode;
   }
}
```

Algorithm to insert a node at a particular position of a SLL

```
struct Node {
   int data;
   struct Node *next;
};
struct Node *head = NULL, *newNode, *temp;
void insertAtPosition(int count) {
   int pos, i = 1;
   printf("Enter the position: ");
   scanf("%d", &pos);
   if (pos > count) {
      printf("Invalid Position\n");
   } else {
      temp = head;
      while (i < pos) {
         temp = temp->next;
         i++;
      }
      newNode = (struct Node *) malloc(sizeof(struct Node));
      printf("Enter data: ");
      scanf("%d", &newNode->data);
      newNode->next = temp->next;
      temp->next = newNode;
   }
}
```

Algorithm to insert a node at a at k-th position/ at a specified position of a SLL

```
struct Node {
    int data;
    struct Node *next;
};
struct Node *head = NULL, *newNode, *temp;
void insertAtPosition() {
    int pos, i = 1;
    printf("Enter position: ");
    scanf("%d", &pos);
    newNode = (struct Node*) malloc(sizeof(struct Node));
    printf("Enter data: ");
    scanf("%d", &newNode->data);
    if (pos == 1) {
        newNode->next = head;
        head = newNode;
    } else {
        temp = head;
        while (i < pos - 1 && temp != NULL) {
            temp = temp->next;
            i++;
        }
        if (temp == NULL) {
            printf("Invalid position");
        } else {
            newNode->next = temp->next;
            temp->next = newNode;
        }
    }
}
```

Node Deletion Operation in Singly Linked List (SLL)

5. Delete from Beginning.
6. Delete from End.
7. Delete from a Particular Position/ After a Given Location.

<u>Algorithm to delete a node from the beginning of a SLL</u>

```
struct Node {
   int data;
   struct Node *next;
};
struct Node *head = NULL;
void deleteFromBeg() {
   struct Node *temp;
   temp = head;
   head = head->next;
   free(temp);
}
```

Algorithm to delete a node from the end of a SLL

```
struct Node {
    int data;
    struct Node *next;
};
struct Node *head = NULL;
void deleteFromEnd() {
    struct Node *temp, *prevNode;
    if (head == NULL) {
        printf("List is empty.\n");
        return;
    }
    temp = head;
    while (temp->next != NULL) {
        prevNode = temp;
        temp = temp->next;
    }
    if (temp == head) {
        head = NULL;
    } else {
        prevNode->next = NULL;
    }
    free(temp);
}
```

Algorithm to delete a node from a specified position of a SLL

```
struct Node {
   int data;
   struct Node *next;
};
struct Node *head = NULL;
void deleteFromPosition() {
   struct Node *temp, *nextNode;
   int pos, i = 1;
   temp = head;
   printf("Enter position: ");
   scanf("%d", &pos);
   while (i < pos - 1) {
      temp = temp->next;
      i++;
   }
   nextNode = temp->next;
   temp->next = nextNode->next;
   free(nextNode);
}
```

What is the difference between Linked List and Linear Array?

S. No.	Array	Linked List
1.	Insertions and deletions are difficult.	Insertions and deletions can be done easily.
2.	It needs movements of elements for insertion and deletion	It does not need movement of nodes for insertion and deletion.
3.	In it space is wasted	In it space is not wasted.
4.	It is more expensive	It is less expensive.
5.	It requires less space as only information is stored	It requires more space as pointers are also stored along with information.
6.	Its size is fixed	Its size is not fixed.
7.	It cannot be extended or reduced according to requirements	It can be extended or reduced according to requirements.
8.	Same amount of time is required to access each element	Different amount of time is required to access each element.
9.	Elements are stored in consecutive memory locations.	Elements may or may not be stored in consecutive memory locations.
10.	If have to go to a particular element then we can reach their directly.	If we have to go to a particular node then we have to go through all those nodes that come before that node.

Advantages of Linked Lists

Below are some of the advantages of linked lists:

- Dynamic size
- Efficient insertions and deletions
- No memory waste
- Flexibility in memory allocation
- Easy implementation of abstract data types
- Flexibility in rearrangement

Disadvantages of Linked Lists

Below are some of the disadvantages of linked lists:

- Random access is inefficient
- Extra memory usage
- Reverse traversal is difficult
- Cache performance
- No binary search
- Vulnerable to memory leaks

Applications of Linked List

Linked lists are used in many areas where dynamic memory and flexible data structures are needed.

- Dynamic memory allocation
- Implementation of stack and queue
- Polynomial representation
- Sparse matrix representation
- Undo/redo operations
- Navigation (forward/backward traversal)
- Graph representation (adjacency list)
- File system (linked file allocation)